

QVerse — Week 14: Noise, Transpilation, and Real Hardware

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `14_noise_transpilation_hardware.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 14; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Explain why ideal and hardware circuits differ.
- Transpile circuits for a restricted target.
- Compare depth and operation counts under optimization.
- Simulate a simple noisy Bell experiment when Qiskit Aer is available.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. Logical circuits versus physical hardware

A circuit drawn in a notebook is a **logical circuit**. It specifies what transformation we want.

A real processor imposes additional constraints:

- only certain native gates are available;
- only certain qubit pairs can interact directly;
- gates have finite error rates;
- qubits decohere;
- measurements are imperfect;
- calibrations vary with time.

Therefore the logical circuit must be translated into instructions that a specific backend can execute.

That translation is **transpilation**.

2. Basis gates

A backend has a set of native or preferred operations called a **basis gate set**.

A high-level gate may need to be decomposed into these primitives.

If the target basis is, for example,

$$\{R_z, SX, X, CX\},$$

then a circuit containing other gates must be rewritten using equivalent combinations of these operations.

Different hardware technologies expose different native gates.

3. Connectivity and coupling maps

Many devices do not allow a two-qubit gate between every pair of qubits.

A **coupling map** or connectivity graph describes which physical qubits can interact directly.

For a linear three-qubit topology,

$$0 - 1 - 2,$$

a direct interaction between physical qubits 0 and 2 is not natively available.

The compiler must route information through intermediate qubits or choose a different logical-to-physical mapping.

4. Routing and SWAP overhead

A SWAP exchanges two qubit states and can be decomposed into three CNOTs in a common gate model.

Routing a logical interaction across a sparse device can therefore add multiple noisy two-qubit operations.

This is why logical connectivity matters.

A circuit with a small abstract gate count can become substantially larger after it is mapped to hardware.

5. Transpilation stages

A modern transpilation workflow generally includes ideas such as:

1. **Unrolling / decomposition:** rewrite high-level gates into supported primitives.
2. **Initial layout:** assign logical qubits to physical qubits.
3. **Routing:** add operations required by limited connectivity.
4. **Optimization:** cancel or simplify redundant gates.
5. **Scheduling / backend-specific processing:** respect timing constraints when needed.

Exact compiler internals can change, but these conceptual stages remain useful.

6. Optimization levels

Qiskit's transpiler exposes several optimization levels.

Conceptually:

- low levels prioritize speed and minimal compilation effort;
- higher levels search more aggressively for reductions in circuit cost.

A higher optimization level is not guaranteed to improve every metric for every circuit. It may trade compilation time against depth, gate count, or other objectives.

The notebook compares levels 0 through 3 and records metrics rather than assuming which is best.

7. Important circuit metrics

Depth

The number of sequential gate layers along the critical path.

Two-qubit gate count

Often a major predictor of hardware error because entangling gates are usually less accurate than single-qubit gates.

Total operation count

Useful, but not sufficient by itself.

Layout quality

A good mapping can avoid routing overhead.

Execution time

Physical duration also matters because of decoherence.

The “best” circuit depends on the device and the application.

8. Decoherence

Two important time scales are commonly denoted T_1 and T_2 .

Energy relaxation T_1

An excited-state population decays toward the ground state.

A simple model is

$$P_1(t) \sim e^{-t/T_1}.$$

Dephasing T_2

Coherent phase relationships decay.

This destroys interference even if energy populations have not fully relaxed.

Typically

$$T_2 \leq 2T_1$$

for common Markovian models, with additional pure dephasing often reducing T_2 further.

The exact physical meaning depends on hardware technology, but the practical lesson is universal: long circuits accumulate more decoherence.

9. Gate errors

Real gates differ from their ideal unitary transformations.

A simple pedagogical model is **depolarizing noise**. For a one-qubit state,

$$\rho \rightarrow (1-p)\rho + p\frac{I}{2}$$

in a simplified form.

For multi-qubit systems, analogous channels mix the desired state with errors over the larger space.

Depolarizing noise is not a complete physical description, but it is useful for seeing how increasing gate count degrades output quality.

10. Readout error

Even if a qubit reaches the correct final state, the classical measurement device may misclassify the result.

A simple readout confusion matrix is

$$M = \begin{pmatrix} P(0|0) & P(0|1) \\ P(1|0) & P(1|1) \end{pmatrix}.$$

Nonzero off-diagonal entries represent misclassification.

Readout error affects observed histograms independently of coherent gate evolution.

11. Ideal versus noisy Bell experiment

For an ideal Bell state $|\Phi^+\rangle$,

$$P(00) = P(11) = \frac{1}{2},$$

and undesired outcomes 01, 10 have zero probability.

Under gate and readout noise, the histogram may contain nonzero counts in all four outcomes.

A simple error metric is

$$\epsilon_{\text{undesired}} = \frac{n_{01} + n_{10}}{N}.$$

This does not capture every aspect of state quality, but it provides an intuitive quantitative comparison.

12. Error mitigation versus error correction

Error mitigation attempts to reduce bias in estimated results without fully encoding logical quantum information in a fault-tolerant code.

Examples can include:

- readout mitigation;
- zero-noise extrapolation;
- probabilistic error cancellation;
- symmetry checks.

Quantum error correction encodes logical qubits redundantly so errors can be detected and corrected during computation.

Mitigation does not create a perfect logical qubit and is not equivalent to fault tolerance.

13. Why mathematically equivalent circuits can perform differently

Two circuits may implement the same ideal unitary but have different:

- depths;
- numbers of two-qubit gates;
- routing requirements;
- idle times;
- exposure to noisy operations.

Therefore equivalence in ideal mathematics does not imply equal hardware fidelity.

This is the central engineering lesson of Week 14.

14. Reproducible hardware experiments

A serious hardware report should record:

- backend name;
- execution date;
- shot count;
- circuit before and after transpilation;
- layout if relevant;
- optimization settings;
- software versions;
- calibration context when available;
- random/transpiler seeds when useful.

Hardware changes over time, so the same circuit may not produce identical results on another day.

15. Current cloud workflow concept

The exact cloud API can evolve. The stable conceptual workflow is:

1. select an accessible backend;
2. inspect its properties and constraints;
3. transpile to the backend's instruction set architecture;
4. execute using the current runtime primitive;
5. retrieve sampled or expectation-value results;
6. compare with ideal expectations;
7. document the backend and execution settings.

The course intentionally treats account setup as an optional extension rather than a core requirement.

16. Common misconceptions

“Transpilation changes the algorithm”

A correct transpilation preserves the intended computation while changing its physical implementation.

“Optimization level 3 is always best”

Not necessarily for every metric, circuit, or backend.

“More shots remove hardware noise”

They reduce statistical uncertainty but do not eliminate systematic errors.

“Error mitigation is error correction”

They solve different levels of the problem.

17. What to know before the notebook

You should be able to explain:

- basis gates;
- coupling maps;
- routing;
- why SWAP overhead is costly;
- depth and two-qubit gate count;
- gate, decoherence, and readout errors;
- the distinction between mitigation and correction;
- why hardware experiments must record backend and compilation metadata.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.